Port Technical Assignment: Agentic Bug Triag & Resolution

System Review

The system is built to triage and solve real issues reported on GitHub for a specific project.
Main handling process:

1. Webhook server - connecting specific GitHub events using ngrok to trigger the system
2. **Orchestrator**:
    a. Fetching the context from GitHub and checking for dedup (issue already handled).
    b. Call **Triage** agent:
        i. Parsing reproducing steps (Haiku)
        ii. Reproduce the bug (Haiku + tools) - limited to 25 iterations, will utilize curl for BE issues, and Playwright for FE ones.
            1. Check reproduction successfully - was the agent actually able to reproduce the bug (true/false, and *reproduction confidence score*). Inability to reproduce will cause the process to be stopped.
        iii. Validate is bug (Haiku + tools) - using the above and the documentation for expected behavior (mocked)
        iv. Root cause analysis (Haiku + tools) - identify the possible source of the issue and define certainty.
            1. In case of low certainty, retry with Opus.
        v. Find relevant people to tag (APIs) - experts who contributed the most, and the one who caused the bug - mocked (since it tagged real people)
        vi. Severity classification (Haiku) - Define the *severity* of the issue
        vii. Comment posting to GitHub and notify Slack
    c. Get back data, store in context
    d. Call **Solve** agent:
        i. Create fix branch (cli)
        ii. Develop and apply the fix (Haiku) - locate the issue, search previous commits, make minimal changes, and verify by building and testing.
        iii. Verify - re-run the reproduction steps against the code, using curl for BE and utilize Playwright for FE. Define the *verification confidence.*
            1. In case of low confidence, retry with Opus
        iv. Capture diff (cli)
        v. Evaluate Risk (Haiku and rule-based) - define the *risk* of making the change.
            1. Make a level-of-autonomy decision: either generate a PR or wait for human feedback.
                a. High risk, Critical severity - HITL
                b. Low–High risk - HITL
                c. Low risk - create PR

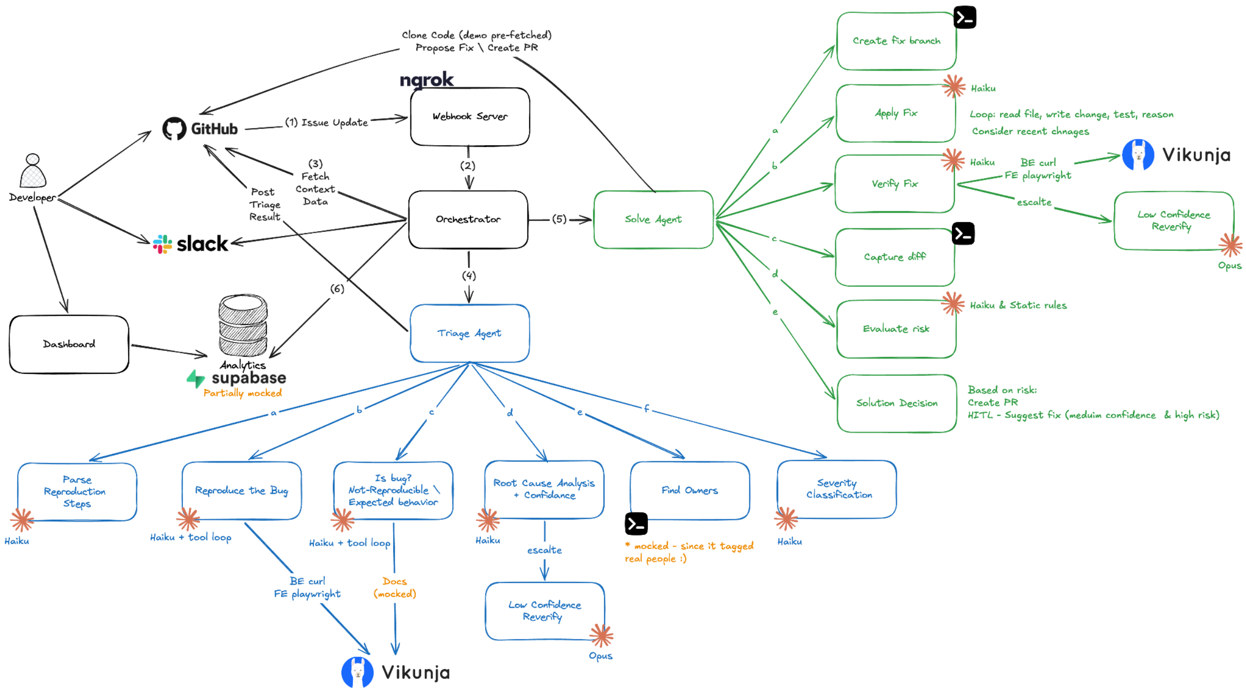
    e. Evaluate the cost of the process using a tracker that counts tokens per model throughout the process.
    f. Comment posting to GitHub and notify Slack
    g. Update data for monitoring and observability
        i. This will be available in the Dashboard for the agent developers

Basic Benchmarking

	BE Bug	BE with Opus	FE Bug	FE with Playwright
Avg time	90s	135s	73s	103s
Token in	88k	108k Haiku 36k Opus	93k	200k
Tokens out	7k	5k Hiku 3k Opus	8k	6k
Avg cost	$0.12	$0.4	$0.14	$0.23

Next Steps

1. Have the system running remotely;
    a. This will also require cloning the code on demand
    b. This might also complicate the usage of Playwright
    c. This will allow handling scale and improving stability
2. Listening to more GitHub integrations and understanding the complete resolution/rejection cycle.
3. Add more data to the monitoring and improve the dashboards. Adding thresholds and alerts on system stability or performance degradations.
4. Posting proof images and recording to an image-storing service and sharing them as links
5. Connect to documentation with MCP to identify expected behavior (not a bug)
6. Better handling of issues in parallel
7. Better learning system which improves and learns with time
8. Centralize notification handling in a single place
9. Better monitoring and handling of usage budget
10. Extend the break conditions under which the models will decide not to make a fix, based on validation and certainty.


Clone Code (demo pre-fetched)
Propose Fix \ Create PR
ngrok
GitHub
(1) Issue Update
Webhook Server
(2)
(3)
Fetch
Context
Data
Post
Triage
Result
Developer
slack
Orchestrator
(5)
Solve Agent
(4)
(6)
Dashboard
Analytics
supabase
Partially mocked
Triage Agent
Create fix branch
Apply Fix
Haiku
Loop: read file, write change, test, reason
Consider recent chnages
Verify Fix
Haiku
BE curl
FE playwright
Vikunja
escalte
Low Confidence
Reverify
Opus
Capture diff
Evaluate risk
Haiku & Static rules
Solution Decision
Based on risk:
Create PR
HITL - Suggest fix (meduim confidence & high risk)
Parse
Reproduction
Steps
Haiku
Reproduce the Bug
Haiku + tool loop
Is bug?
Not-Reproducible \
Expected behavior
Haiku + tool loop
Root Cause Analysis
+ Confidance
Haiku
escalte
Low Confidence
Reverify
Opus
Find Owners
* mocked - since it tagged
real people :)
Severity
Classification
Haiku
BE curl
FE playwright
Docs
(mocked)
Vikunja

[link]